MySQL Project • Theme, ER Diagram, Schema, Sample Data and SQL Queries

1. Theme

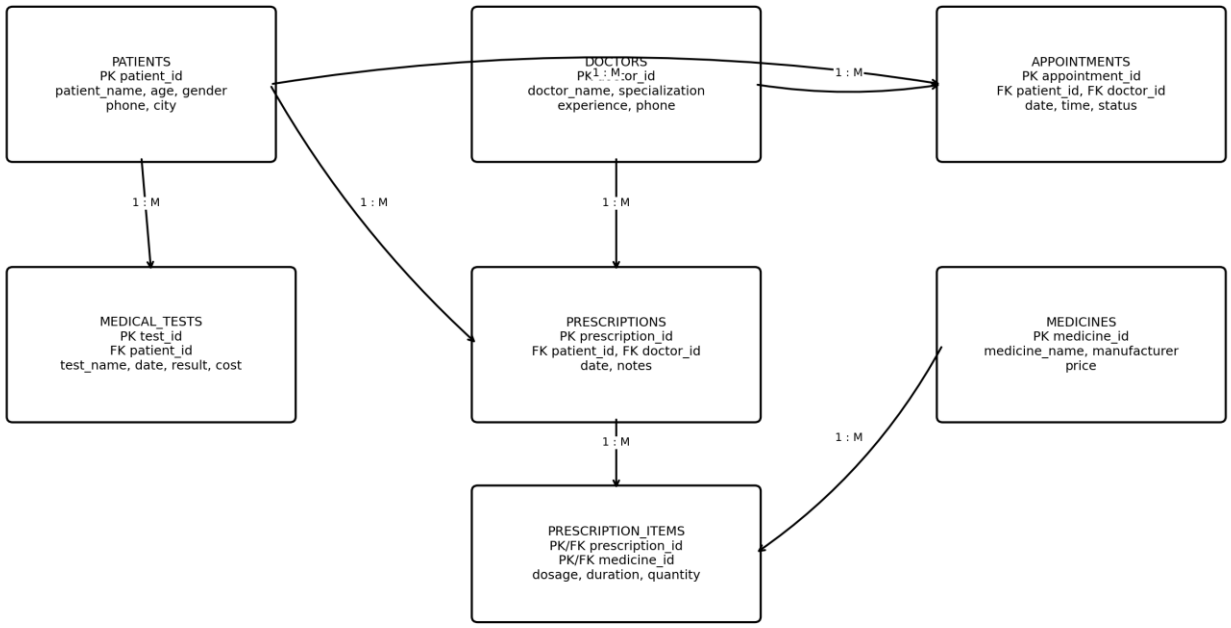
A cardiology department wants to maintain patient and doctor details, appointments, medical tests, prescriptions and medicines. This database helps organize hospital information and retrieve appointment, test, prescription and medicine reports using SQL.

2. Modules

- Patients — stores patient details such as name, age, gender, phone and city.
- Doctors — stores doctor name, specialization, experience and phone.
- Appointments — links patients with doctors and records date, time and status.
- Medical Tests — stores test name, date, result and cost for each patient.
- Prescriptions — records prescriptions issued by doctors to patients.
- Medicines — stores medicine names, manufacturers and prices.
- Prescription Items — links medicines to prescriptions and stores quantity and dosage notes.

3. ER Diagram

The database contains 7 tables and 7 one-to-many relationships. PK means Primary Key; FK means Foreign Key.



4. Relationships

Parent	Child	Type	Key
patients	appointments	1 : M	patient_id
doctors	appointments	1 : M	doctor_id
patients	medical_tests	1 : M	patient_id
patients	prescriptions	1 : M	patient_id
doctors	prescriptions	1 : M	doctor_id
prescriptions	prescription_items	1 : M	prescription_id
medicines	prescription_items	1 : M	medicine_id

5. Table Structure

Table	Columns
patients	PK patient_id, patient_name, age, gender, phone, city
doctors	PK doctor_id, doctor_name, specialization, experience, phone
appointments	PK appointment_id, FK patient_id, FK doctor_id, appointment_date, appointment_time, status
medical_tests	PK test_id, FK patient_id, test_name, test_date, result, cost

prescriptions	PK prescription_id, FK patient_id, FK doctor_id, prescription_date, notes
medicines	PK medicine_id, medicine_name, manufacturer, price
prescription_items	Composite PK (prescription_id, medicine_id); both are FKs; dosage, duration_days, quantity

6. Schema (DDL)

```
CREATE DATABASE IF NOT EXISTS cardiology_db;
USE cardiology_db;

# Table 1: Patients
CREATE TABLE IF NOT EXISTS patients (
    patient_id INT PRIMARY KEY,
    patient_name VARCHAR(100) NOT NULL,
    age INT,
    gender VARCHAR(10),
    phone VARCHAR(15),
    city VARCHAR(50)
) ENGINE=InnoDB;

# Table 2: Doctors
CREATE TABLE IF NOT EXISTS doctors (
    doctor_id INT PRIMARY KEY,
    doctor_name VARCHAR(100) NOT NULL,
    specialization VARCHAR(100),
    experience INT,
    phone VARCHAR(15)
) ENGINE=InnoDB;

# Table 3: Appointments
CREATE TABLE IF NOT EXISTS appointments (
    appointment_id INT PRIMARY KEY,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
    appointment_date DATE NOT NULL,
```

```
    appointment_time TIME,
    status VARCHAR(20) DEFAULT 'Scheduled',
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
    FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id)
) ENGINE=InnoDB;

# Table 4: Medical Tests
CREATE TABLE IF NOT EXISTS medical_tests (
    test_id INT PRIMARY KEY,
    patient_id INT NOT NULL,
    test_name VARCHAR(100) NOT NULL,
    test_date DATE,
    result VARCHAR(255),
    cost DECIMAL(10,2),
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id)
) ENGINE=InnoDB;

# Table 5: Prescriptions
CREATE TABLE IF NOT EXISTS prescriptions (
    prescription_id INT PRIMARY KEY,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
    prescription_date DATE NOT NULL,
    notes VARCHAR(255),
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
    FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id)
) ENGINE=InnoDB;

# Table 6: Medicines
CREATE TABLE IF NOT EXISTS medicines (
    medicine_id INT PRIMARY KEY,
    medicine_name VARCHAR(100) NOT NULL,
    manufacturer VARCHAR(100),
    price DECIMAL(10,2)
) ENGINE=InnoDB;

# Table 7: Prescription Items
```

```
CREATE TABLE IF NOT EXISTS prescription_items (
    prescription_id INT NOT NULL,
    medicine_id INT NOT NULL,
    dosage VARCHAR(100),
    duration_days INT,
    quantity INT,
    PRIMARY KEY (prescription_id, medicine_id),
    FOREIGN KEY (prescription_id) REFERENCES
prescriptions(prescription_id),
    FOREIGN KEY (medicine_id) REFERENCES medicines(medicine_id)
) ENGINE=InnoDB;

SHOW TABLES;
```

7. Sample Data

Run these INSERT statements after creating the tables. The IDs are fixed so that foreign-key relationships are easy to understand. If you already used these IDs, run the inserts only in an empty project database or adapt the records to avoid duplicate-key errors.

```
# Patients
INSERT INTO patients VALUES
(1, 'Rahul Kumar', 45, 'Male', '9876501001', 'Srikakulam'),
(2, 'Priya Sharma', 38, 'Female', '9876501002', 'Visakhapatnam'),
(3, 'Arjun Reddy', 55, 'Male', '9876501003', 'Vijayawada'),
(4, 'Sneha Rao', 42, 'Female', '9876501004', 'Hyderabad'),
(5, 'Kiran Kumar', 60, 'Male', '9876501005', 'Srikakulam');

# Doctors
INSERT INTO doctors VALUES
(1, 'Dr. Suresh Reddy', 'Cardiologist', 15, '9888001001'),
(2, 'Dr. Anitha Rao', 'Interventional Cardiologist', 12,
'9888001002'),
(3, 'Dr. Rajesh Kumar', 'Cardiac Surgeon', 18, '9888001003'),
(4, 'Dr. Lakshmi Devi', 'Cardiologist', 10, '9888001004');

# Appointments
```

```
INSERT INTO appointments VALUES
(1001, 1, 1, '2026-11-02', '09:30:00', 'Scheduled'),
(1002, 2, 2, '2026-11-02', '10:00:00', 'Scheduled'),
(1003, 3, 3, '2026-11-03', '11:00:00', 'Completed'),
(1004, 1, 1, '2026-11-04', '12:00:00', 'Scheduled'),
(1005, 4, 4, '2026-11-05', '09:00:00', 'Cancelled');

# Medical Tests (illustrative sample data)
INSERT INTO medical_tests VALUES
(1, 1, 'ECG', '2026-10-20', 'Example result; clinician review
required', 500.00),
(2, 2, 'Echocardiogram', '2026-10-21', 'Example result; clinician
review required', 2500.00),
(3, 3, 'Lipid Profile', '2026-10-22', 'Example result; clinician
review required', 800.00),
(4, 4, 'Stress Test', '2026-10-23', 'Example result; clinician
review required', 1800.00),
(5, 5, 'ECG', '2026-10-24', 'Example result; clinician review
required', 500.00);

# Prescriptions (academic sample records)
INSERT INTO prescriptions VALUES
(501, 1, 1, '2026-10-20', 'Sample record only; follow clinician
instructions'),
(502, 2, 2, '2026-10-21', 'Sample record only; follow clinician
instructions'),
(503, 3, 3, '2026-10-22', 'Sample record only; follow clinician
instructions'),
(504, 4, 4, '2026-10-23', 'Sample record only; follow clinician
instructions'),
(505, 5, 1, '2026-10-24', 'Sample record only; follow clinician
instructions');

# Medicines
INSERT INTO medicines VALUES
(101, 'Aspirin', 'Example Pharma A', 50.00),
(102, 'Atorvastatin', 'Example Pharma B', 120.00),
```

```
(103, 'Metoprolol', 'Example Pharma C', 90.00),
(104, 'Amlodipine', 'Example Pharma D', 75.00),
(105, 'Clopidogrel', 'Example Pharma E', 110.00);

# Prescription Items (dosage text is a placeholder, not medical
advice)
INSERT INTO prescription_items VALUES
(501, 101, 'As prescribed by clinician', 5, 5),
(501, 102, 'As prescribed by clinician', 30, 30),
(502, 103, 'As prescribed by clinician', 7, 14),
(503, 104, 'As prescribed by clinician', 10, 10),
(504, 105, 'As prescribed by clinician', 15, 15),
(505, 102, 'As prescribed by clinician', 30, 30);
```

8. SQL Queries

1. Display all patients

```
SELECT * FROM patients;
```

2. Display all doctors

```
SELECT * FROM doctors;
```

3. Display all medicines

```
SELECT * FROM medicines;
```

4. Find total number of patients

```
SELECT COUNT(*) AS total_patients FROM patients;
```

5. Find total number of doctors

```
SELECT COUNT(*) AS total_doctors FROM doctors;
```

6. Find total number of appointments

```
SELECT COUNT(*) AS total_appointments FROM appointments;
```

7. Display appointment details with patient and doctor names

```
SELECT a.appointment_id, p.patient_name, d.doctor_name,
       a.appointment_date, a.appointment_time, a.status
FROM appointments a
JOIN patients p ON a.patient_id = p.patient_id
```

```
JOIN doctors d ON a.doctor_id = d.doctor_id
ORDER BY a.appointment_date, a.appointment_time;
```

8. Find scheduled appointments

```
SELECT * FROM appointments WHERE status = 'Scheduled';
```

9. Find cancelled appointments

```
SELECT * FROM appointments WHERE status = 'Cancelled';
```

10. Find patients from Srikakulam

```
SELECT * FROM patients WHERE city = 'Srikakulam';
```

11. Find patients aged 50 or above

```
SELECT * FROM patients WHERE age >= 50;
```

12. Count patients in each city

```
SELECT city, COUNT(*) AS total_patients FROM patients GROUP BY
city;
```

13. Find patients who have no appointments

```
SELECT p.patient_id, p.patient_name
FROM patients p LEFT JOIN appointments a ON p.patient_id =
a.patient_id
WHERE a.appointment_id IS NULL;
```

14. Count appointments for each doctor

```
SELECT d.doctor_name, COUNT(a.appointment_id) AS
appointment_count
FROM doctors d LEFT JOIN appointments a ON d.doctor_id =
a.doctor_id
GROUP BY d.doctor_id, d.doctor_name ORDER BY appointment_count
DESC;
```

15. Display medical tests with patient names

```
SELECT p.patient_name, mt.test_name, mt.test_date, mt.result,
mt.cost
FROM medical_tests mt JOIN patients p ON mt.patient_id =
p.patient_id
ORDER BY mt.test_date;
```

16. Calculate test costs for each patient

```
SELECT p.patient_name, SUM(mt.cost) AS total_test_cost
FROM patients p JOIN medical_tests mt ON p.patient_id =
mt.patient_id
GROUP BY p.patient_id, p.patient_name;
```

17. Calculate total medical test cost

```
SELECT SUM(cost) AS total_test_cost FROM medical_tests;
```

18. Display prescription details with patient and doctor

```
SELECT pr.prescription_id, p.patient_name, d.doctor_name,
pr.prescription_date, pr.notes
FROM prescriptions pr JOIN patients p ON pr.patient_id =
p.patient_id
JOIN doctors d ON pr.doctor_id = d.doctor_id;
```

19. Display medicines listed in each prescription

```
SELECT pr.prescription_id, p.patient_name, m.medicine_name,
pi.dosage,
       pi.duration_days, pi.quantity, m.price
FROM prescription_items pi JOIN prescriptions pr ON
pi.prescription_id = pr.prescription_id
JOIN patients p ON pr.patient_id = p.patient_id
JOIN medicines m ON pi.medicine_id = m.medicine_id
ORDER BY pr.prescription_id, m.medicine_name;
```

20. Count prescription items for each medicine

```
SELECT m.medicine_name, COUNT(pi.medicine_id) AS times_listed
FROM medicines m LEFT JOIN prescription_items pi ON m.medicine_id
= pi.medicine_id
GROUP BY m.medicine_id, m.medicine_name ORDER BY times_listed
DESC;
```

21. Calculate example medicine value per prescription

```
SELECT pr.prescription_id, p.patient_name, SUM(pi.quantity *
m.price) AS example_medicine_value
FROM prescriptions pr JOIN patients p ON pr.patient_id =
p.patient_id
```

```
JOIN prescription_items pi ON pr.prescription_id =
pi.prescription_id
JOIN medicines m ON pi.medicine_id = m.medicine_id
GROUP BY pr.prescription_id, p.patient_name;
```

22. Find the most expensive medicine

```
SELECT * FROM medicines ORDER BY price DESC LIMIT 1;
```

23. Find medicines priced between ₹75 and ₹120

```
SELECT * FROM medicines WHERE price BETWEEN 75 AND 120;
```

24. Find doctors with 12 or more years of experience

```
SELECT * FROM doctors WHERE experience >= 12;
```

25. Find completed appointments

```
SELECT * FROM appointments WHERE status = 'Completed';
```

9. How to Run in MySQL Workbench

1. Connect to your MySQL server and open a new SQL tab.
2. Execute Section 6 (DDL) first to create the database and tables.
3. Execute Section 7 (sample data) next, in the order provided.
4. Run the queries in Section 8 one by one or select a query before clicking Execute.
5. If you already inserted the same fixed IDs, duplicate-primary-key errors may occur. Use a new empty schema for the sample data, or adapt the IDs. Do not drop tables if you need to preserve existing records.
6. CREATE TABLE IF NOT EXISTS does not change the structure of tables that already exist. If an existing table differs, inspect it using DESCRIBE table_name;.